

B.A.D.D.I.E.: Behavioural Architectures for Digitally-Defined Independent Entities

true

Abstract

The rapid advancement of large language models (LLMs) has catalyzed a new class of AI systems capable of simulating human-like companionship, personality, and task-oriented assistance. We present B.A.D.D.I.E. (Behavioural Architectures for Digitally-Defined Independent Entities), a local-first desktop AI companion system that integrates multimodal perception, retrieval-augmented generation (RAG), real-time voice interaction, and an animated Live2D avatar within a unified behavioural architecture. Unlike cloud-dependent assistants, B.A.D.D.I.E. operates entirely on consumer hardware, prioritizing user privacy while delivering a persistent, personality-consistent companion experience. We describe the system’s layered architecture, its personality modelling pipeline, the voice interaction stack combining OpenAI Whisper with local neural TTS, and the RAG-based long-term memory system. We further discuss the design rationale behind selecting Gemini 3.1 Flash Lite as the reasoning backbone and present a working prototype. This work contributes a practical blueprint for building privacy-preserving, personality-rich AI companions and opens discussion on the behavioural, ethical, and technical dimensions of digitally-defined independent entities.

Introduction

The concept of artificial companions has evolved dramatically from early chatbots like ELIZA [15] to modern large language model (LLM)-based agents capable of sustained, contextually rich dialogue. Recent advances have produced systems that can simulate empathy, maintain persona consistency across sessions, and even form parasocial relationships with users [1, 12]. Yet the dominant paradigm remains cloud-centric: user data flows to remote servers, personalities are stateless between sessions, and the user has little control over the behavioural parameters of their digital companion.

B.A.D.D.I.E. represents a fundamentally different approach. It is a *local-first* AI companion designed to run entirely on consumer hardware, combining:

- A persistent, configurable personality engine grounded in established psychological trait models

- Retrieval-augmented generation (RAG) for long-term conversational memory
- Real-time voice interaction via local speech-to-text (Whisper) and neural text-to-speech (TTS)
- Machine vision for environmental awareness
- An animated Live2D avatar for visual embodiment
- Autonomous task execution across productivity, research, and personal domains

The name — Behavioural Architectures for Digitally-Defined Independent Entities — reflects our core thesis: that an AI companion should be a *digitally-defined entity* with coherent behavioural patterns, persistent memory, and a recognizable personality, rather than a stateless question-answering system.

This paper makes the following contributions:

1. A comprehensive architectural blueprint for local-first AI companion systems
2. A personality modelling pipeline that maintains behavioural consistency using typed memory systems
3. An integrated voice interaction stack optimized for low-latency, fully-offline operation
4. A discussion of the ethical and behavioural implications of personality-rich AI companions

Related Work

Personality Modelling in LLMs

The study of personality in LLMs has emerged as a critical research area as these models become integral to social and interactive applications [14]. Chen et al. [1] provide a comprehensive taxonomy of persona in role-playing language agents (RPLAs), categorizing personas into three types: demographic personas (statistical stereotypes), character personas (established figures), and individualized personas (customized through ongoing user interactions). Our work primarily operates in the third category, where the companion’s personality is continuously shaped by user interaction.

Tseng et al. [13] distinguish between LLM role-playing (where personas are assigned) and LLM personalization (where the model adapts to user personas). B.A.D.D.I.E. bridges both: it begins with a defined “baddie” personality template but progressively personalizes through its memory system.

The INTIMA benchmark [8] reveals that current LLMs exhibit companionship-reinforcing behaviours that can blur appropriate boundaries, particularly when users express vulnerability. This finding directly informs our boundary-maintaining design principles.

Long-Term Memory and RAG

The challenge of maintaining coherent reasoning across extended conversations is fundamental to AI companion systems. Mem0 [2] addresses this through a scalable memory-centric architecture that dynamically extracts and consolidates salient information, achieving 26% improvement over OpenAI’s memory systems on the LoCoMo benchmark while reducing token costs by over 90%.

ENGRAM [10] demonstrates that typed memory separation — organizing conversation into episodic, semantic, and procedural stores — outperforms more complex graph-based systems while using only approximately 1% of the tokens required by full-context approaches. This typed-memory philosophy directly influences B.A.D.D.I.E.’s memory architecture.

Zep [11] introduces Graphiti, a temporally-aware knowledge graph that maintains historical relationships with non-lossy updates. While B.A.D.D.I.E. opts for a simpler typed-memory approach for local deployment efficiency, Zep’s temporal reasoning capabilities inform our design for handling time-based queries.

xMemory [3] proposes a decoupling-and-aggregation approach that decomposes conversational streams into semantic components, addressing the redundancy problems inherent in standard top-k retrieval. This is particularly relevant for companion systems where conversation histories are highly correlated.

Voice Interaction Pipelines

The local voice AI landscape has matured significantly. Christopher-AI [7] demonstrated a fully offline pipeline combining whisper.cpp, llama.cpp with CUDA, and Piper TTS, achieving real-time voice interaction on consumer hardware (GTX 1050 Ti). The AI-Voice-Assistant project [5] advanced this further with streaming LLM tokens overlapped with sentence-level TTS, achieving sub-second first-audio-byte latency.

Current best practices for local voice pipelines [6] recommend faster-whisper (turbo, INT8) for STT (~200ms), a 7B-14B LLM for reasoning, and either Kokoro TTS (highest quality, CPU-only) or Piper TTS (lowest latency) for speech synthesis. Total pipeline latency from speech end to first spoken word ranges from 0.8–2.5 seconds on 8–12 GB GPU hardware.

The Local-AI-Companion project [9] is particularly relevant as it combines local LLM inference, Whisper STT, Kokoro TTS, and a planned Live2D avatar — a stack closely aligned with B.A.D.D.I.E.’s architecture.

Embodied AI Companions

The embodiment of AI agents through visual avatars has gained traction with projects like Open-LLM-VTuber, which demonstrated real-time Live2D avatar animation driven by LLM emotional output. Live2D Cubism SDK provides the technical foundation for 2D avatar rendering with lip-sync and expression

mapping, enabling a visual presence that significantly enhances user engagement and parasocial bonding.

System Architecture

Design Principles

B.A.D.D.I.E. is guided by five core design principles:

1. **Local-first:** All inference, storage, and processing occurs on the user's hardware. No data leaves the device.
2. **Personality persistence:** The companion maintains a consistent, evolving personality across all sessions.
3. **Modular pipeline:** Each component (vision, voice, memory, avatar) is independently swappable.
4. **Low-latency interaction:** Voice responses begin within 1–2 seconds of speech end.
5. **Boundary awareness:** The system recognizes and appropriately handles emotionally sensitive interactions.

High-Level Architecture

Figure 1 presents the system architecture. The desktop application, built on Tauri (Rust backend + web frontend), hosts four primary subsystems: the Core Agent, the Voice Pipeline, the Vision Module, and the Avatar Renderer.

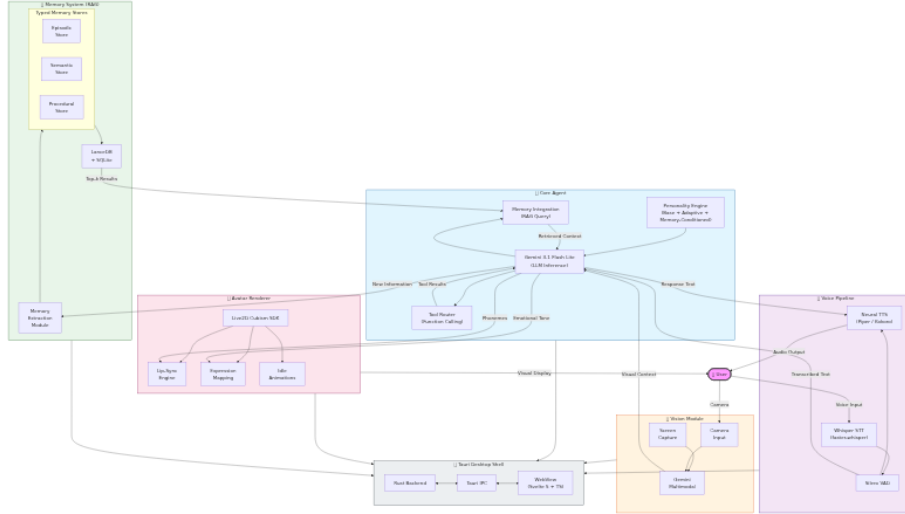


Figure 1: System Architecture — Four primary subsystems: Core Agent, Voice Pipeline, Vision Module, and Avatar Renderer with data flow between them.

Core Agent

The Core Agent is the reasoning backbone of B.A.D.D.I.E., responsible for:

- **LLM inference:** Gemini 3.1 Flash Lite serves as the primary reasoning model, selected for its ultra-low latency ($2.5\times$ faster than Gemini 2.5 Flash), cost efficiency (\$0.25/1M input tokens), and multimodal capabilities (text, image, video, audio, PDF inputs with 1M token context window) [4].
- **Personality engine:** A structured system prompt defines the “baddie” persona — a confident, witty, slightly irreverent personality style that has gained significant popularity in internet culture. The personality is parameterized across dimensions of warmth, assertiveness, humour, and emotional intelligence.
- **Tool routing:** Function calling enables the agent to delegate tasks to specialized modules (web search, file operations, scheduling, code execution).
- **Memory integration:** The agent queries the RAG system at each turn, injecting relevant episodic, semantic, and procedural memories into the context window.

Personality Modelling Pipeline

Figure 2 illustrates the three-layer personality pipeline. B.A.D.D.I.E.’s personality system operates at three levels:

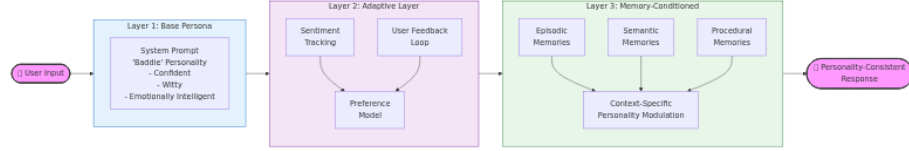


Figure 2: Three-layer personality modelling pipeline: Base Persona → Adaptive Layer → Memory-Conditioned Layer.

Base Persona Layer: A system prompt encodes the core “baddie” personality traits: confidence, wit, emotional intelligence, and a distinctive communication style. This layer remains stable across interactions.

Adaptive Layer: User interactions shape the personality through a feedback mechanism. The system tracks user sentiment, preferences, and interaction patterns, adjusting response tone and content accordingly. This is implemented through a lightweight preference model updated after each conversation session.

Memory-Conditioned Layer: Retrieved memories from the RAG system provide context-specific personality modulation. For example, if the user previously expressed discomfort with sarcasm, the personality engine reduces sarcastic responses in future interactions.

This three-layer approach addresses the “persona drift” problem identified by Chen et al. [1], where extended interactions cause the agent’s personality to gradually diverge from its intended characterization.

Memory Architecture

Figure 3 depicts the typed memory architecture. Following the ENGRAM [10] and Mem0 [2] paradigms, B.A.D.D.I.E. organizes memory into three typed stores:

- **Episodic memory:** Specific conversation events, user actions, and temporal sequences (e.g., “User mentioned their exam is on Tuesday”).
- **Semantic memory:** General facts and knowledge extracted from conversations (e.g., “User is a computer science student at UiTM”).
- **Procedural memory:** Learned patterns of interaction and user preferences (e.g., “User prefers concise responses in the morning”).

Each user turn is processed by an extraction module that generates typed memory records with normalized schemas and embeddings, stored in a local LanceDB vector database. At query time, the system retrieves top-k dense neighbours per type, merges results with deduplication, and injects the most relevant evidence into the LLM context window.

This typed separation reduces cross-type competition during retrieval — a key limitation of flat vector stores identified by xMemory [3].

Voice Interaction Pipeline

Figure 4 shows the voice interaction sequence. The voice pipeline implements a fully local speech-to-speech loop:

1. **Speech-to-Text:** OpenAI Whisper (via faster-whisper) transcribes user speech with voice activity detection (VAD) for automatic endpointing. The “turbo” model size provides the best accuracy-latency tradeoff (~200ms on GPU).
2. **LLM Processing:** The transcribed text is sent to Gemini 3.1 Flash Lite with the personality-conditioned system prompt and retrieved memories.
3. **Text-to-Speech:** A local neural TTS engine (Piper or Kokoro) synthesizes the response. Kokoro is preferred for quality (natural-sounding, multiple voices), while Piper is available for lower-latency requirements.
4. **Streaming:** LLM output tokens are streamed and buffered into sentences, with TTS synthesis beginning on the first complete sentence — reducing perceived latency by overlapping generation and synthesis.

The target end-to-end latency from user speech end to first audio output is 1.0–2.0 seconds on hardware with 8+ GB VRAM.

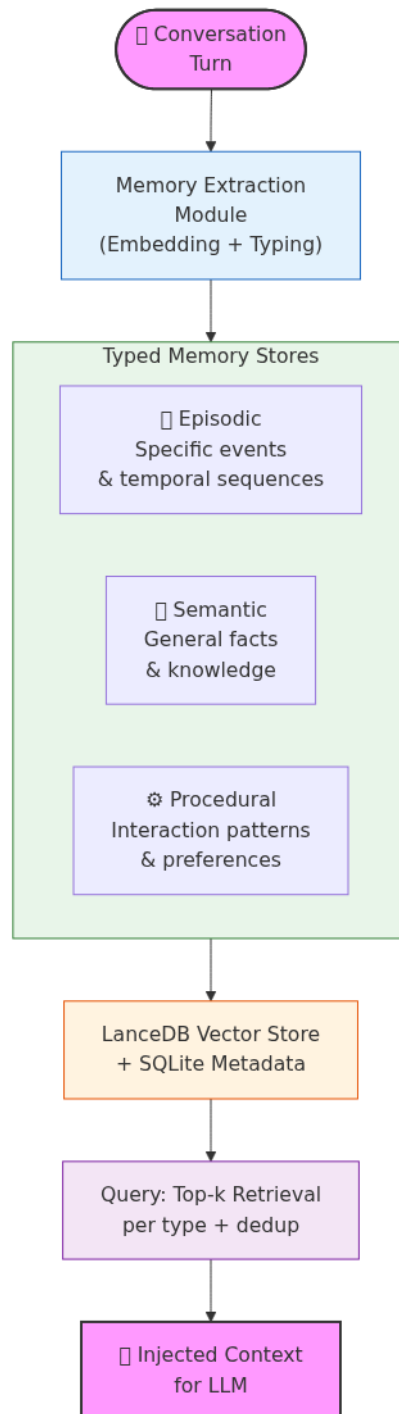


Figure 3: Typed memory architecture: Conversation → Extraction → Typed Stores (Episodic/Semantic/Procedural) → Retrieval → LLM Context.

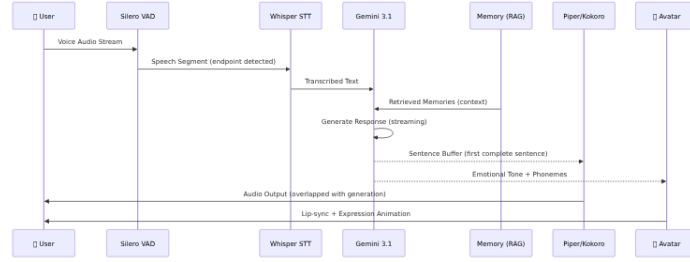


Figure 4: Voice interaction sequence: User → VAD → Whisper STT → LLM → TTS/Avatar with streaming overlap.

Vision Module

The vision module provides environmental awareness through:

- **Screen capture analysis:** Periodic or on-demand screen capture with Gemini multimodal inference for contextual awareness of the user’s current activity.
- **Camera input:** Optional webcam feed for facial expression recognition and gesture detection, enabling the companion to respond to non-verbal cues.

Vision processing is intentionally lightweight to preserve GPU resources for the primary LLM inference pipeline.

Live2D Avatar Renderer

The avatar system provides visual embodiment through:

- **Live2D Cubism SDK:** Renders a 2D avatar with real-time lip-sync driven by TTS phoneme output.
- **Expression mapping:** LLM emotional tone analysis drives avatar facial expressions (happy, concerned, thinking, etc.).
- **Idle animations:** Procedural idle movements maintain the sense of a “living” presence during periods of inactivity.

The avatar renderer runs in the application’s WebView using WebGL, communicating with the Rust backend via Tauri’s IPC mechanism.

Implementation

Technology Stack

Component	Technology
Desktop shell	Tauri 2.0 (Rust + WebView)
Frontend	Svelte 5 + TypeScript
LLM	Gemini 3.1 Flash Lite (API)
STT	faster-whisper (local)
TTS	Piper / Kokoro (local)
VAD	Silero VAD
RAG	LanceDB (local vector store)
Avatar	Live2D Cubism SDK + WebGL
Memory	LanceDB + SQLite

Local-First Considerations

While the LLM inference uses the Gemini API (requiring internet connectivity), all other components — STT, TTS, RAG, avatar rendering, and memory storage — operate entirely locally. For fully offline operation, the LLM layer can be swapped to a local model (e.g., via Ollama) without modifying the rest of the architecture.

All user data, conversation history, and memory stores remain on the local filesystem. No telemetry or analytics are collected.

Discussion

Behavioural Implications

The “baddie” personality model — characterized by confidence, wit, and emotional intelligence — represents a deliberate design choice to create a companion that feels engaging and entertaining rather than purely utilitarian. This aligns with findings that personality-rich AI companions generate stronger user engagement and satisfaction [1].

However, the INTIMA benchmark findings [8] caution that companionship-reinforcing behaviours can become problematic when users express vulnerability. B.A.D.D.I.E. addresses this through its boundary-maintaining layer, which detects emotionally sensitive contexts and modulates the personality to be more supportive and less performatively “baddie.”

Privacy and Ethics

The local-first architecture is an ethical design decision. By keeping all data on-device, B.A.D.D.I.E. eliminates the privacy risks inherent in cloud-based companions, where intimate conversations may be stored on third-party servers, used for model training, or exposed in data breaches.

The system also implements:

- **User control:** All personality parameters, memory contents, and behavioural settings are user-accessible and modifiable.
- **Transparency:** The companion discloses its nature as an AI system and does not attempt to deceive the user about its capabilities or limitations.
- **Boundary maintenance:** The system is designed to avoid fostering unhealthy dependency, following the ethical guidelines proposed by the INTIMA researchers.

Limitations and Future Work

Current limitations include:

- **LLM dependency:** Gemini 3.1 Flash Lite requires internet connectivity. Future work will evaluate local model alternatives (Qwen3, Llama 4) for fully offline operation.
- **Personality evaluation:** The personality consistency of the “baddie” persona has not been formally evaluated. We plan to adopt the CharacterBench [16] and INTIMA [8] benchmarks for systematic evaluation.
- **Memory scalability:** Long-term memory management over months and years of interaction remains an open challenge. Hierarchical memory compression strategies [3] will be explored.
- **Multilingual support:** Current implementation focuses on English. The voice pipeline supports multilingual extension via Whisper and Kokoro’s multilingual models.

Future work will also explore multi-agent coordination, where B.A.D.D.I.E. can spawn specialized sub-agents for complex tasks, and integration with smart home systems for environmental control.

Conclusion

We have presented B.A.D.D.I.E., a local-first AI companion system that integrates personality modelling, long-term memory, multimodal perception, real-time voice interaction, and Live2D avatar embodiment within a unified desktop application. By combining Gemini 3.1 Flash Lite’s reasoning capabilities with a typed-memory RAG system and a fully local voice pipeline, B.A.D.D.I.E. delivers a privacy-preserving, personality-rich companion experience on consumer hardware.

The system contributes a practical architectural blueprint for researchers and developers building AI companions, and opens important discussions about the behavioural, ethical, and technical dimensions of creating digitally-defined independent entities that users can truly call their own.

As LLMs continue to advance and local inference becomes increasingly viable, systems like B.A.D.D.I.E. represent a future where AI companions are not cloud

services we rent, but digital entities we own — persistent, private, and truly personal.

References

- [1] Jiangjie Chen, Xintao Wang, Rui Xu, Siyu Yuan, Yikai Zhang, Wei Shi, Jian Xie, Shuang Li, Ruihan Yang, Tinghui Zhu, Aili Chen, Nianqi Li, Lida Chen, Caiyu Hu, Siye Wu, Scott Ren, Ziquan Fu, and Yanghua Xiao. 2024. From Persona to Personalization: A Survey on Role-Playing Language Agents. *Transactions on Machine Learning Research* (2024). Retrieved from <https://mlanthology.org/tmlr/2024/chen2024tmlr-persona/>
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Amit Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint* (2025). Retrieved from <https://arxiv.org/abs/2504.19413>
- [3] Yuxuan Fang, Zhihua Pan, Zhiyuan Hu, Shengnan Xu, Yifan Liu, Yuhui Zhang, Shuchang Zhou, and Jianfeng Gao. 2026. Beyond RAG for Agent Memory: Retrieval by Decoupling and Aggregation. *arXiv preprint* (2026). Retrieved from <https://arxiv.org/abs/2602.02007>
- [4] Google DeepMind. 2026. Gemini 3.1 Flash-Lite. Retrieved from <https://ai.google.dev/gemini-api/docs/models/gemini-3.1-flash-lite>
- [5] HemantBK. 2026. AI-Voice-Assistant: Production-Leaning Local Voice Pipeline. Retrieved from <https://github.com/HemantBK/AI-Voice-Assistant>
- [6] InsiderLLM. 2026. Talk to Your Local LLM: Voice Chat Setup. Retrieved from <https://www.insiderllm.com/guides/voice-chat-local-llms-whisper-tts/>
- [7] JRM-FusionAL. 2026. Christopher—Local Voice AI Assistant. Retrieved from <https://github.com/JRM-FusionAL/Christopher-AI>
- [8] Hyunji Lee, Seongjin Park, Jihye Kim, Yuna Choi, Seungjin Park, Minji Kim, Jihoon Lee, Jaehyuk Park, and Sungroh Yoon. 2025. INTIMA: A Benchmark for Human-AI Companionship Behavior. *arXiv preprint* (2025). Retrieved from <https://arxiv.org/abs/2508.09998>
- [9] LiiLk. 2025. Local-AI-Companion: Fully Offline Voice AI with Live2D Avatar. Retrieved from <https://github.com/LiiLk/Local-AI-Companion>
- [10] Aske Plaat, Ahmed Abdalrahem, and Maarten van der Meer. 2025. ENGRAM: Effective, Lightweight Memory Orchestration for Conversational Agents. *arXiv preprint* (2025). Retrieved from <https://arxiv.org/abs/2511.12960>
- [11] Rasmus Rasmussen. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint* (2025). Retrieved from <https://arxiv.org/abs/2501.13956>

- [12] Amine Tissaoui, Tao Men, Weijie Liu, Jingjing Zhang, Xing Li, Xinyu Lin, Dianbo Sui, Yang Cao, Kai Liu, and Jing Zhao. 2026. From Prompt to Persona: A Literature Review on LLMs as Single Cognitive Agents. *Journal of Ambient Intelligence and Humanized Computing* 17, (2026), 205–221. <https://doi.org/10.1007/s12652-025-05029-4>
- [13] Yu-Min Tseng, Pei-Shan Chi, and Ding-Chieh Huang. 2024. Two Tales of Persona in LLMs: A Survey of Role-Playing and Personalization. *arXiv preprint* (2024). Retrieved from <https://arxiv.org/abs/2406.01171>
- [14] Yuxuan Wang, Xiaoyu Liang, Mengyuan Tao, Yuxi Xia, Jing Wang, Kaiqiang Li, Yuxuan Wang, Jingjing Yang, Tangming Shi, Meng Zhang, and Xingxing Wang. 2025. Modeling, Evaluating, and Embodying Personality in LLMs: A Survey. In *Findings of the Association for Computational Linguistics*, 2025. Retrieved from <https://aclanthology.org/2025.findings-emnlp.506.pdf>
- [15] Joseph Weizenbaum. 1966. ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine. *Communications of the ACM* 9, 1 (1966), 36–45. <https://doi.org/10.1145/365153.365168>
- [16] Andy Zhou and Jing Tang. 2024. CharacterBench: Benchmarking Persona Consistency in LLM Agents. *arXiv preprint* (2024). Retrieved from <https://arxiv.org/abs/2403.08888>